

APGAR

A Pretty Good Auto Router

Architecture Specification

GPU-native candidate generation, global allocation, and exact PCB legalization

Project name

APGAR is intentionally modest in name and ambitious in architecture: a router should earn a high score through measurable connectivity, legality, manufacturability, and route quality rather than claiming universal optimality.

Document	APGAR Architecture Specification
Version	0.1 - Project kickoff baseline
Status	Draft / living specification
Date	July 17, 2026
Primary audience	Router, geometry, GPU, CAD integration, and verification engineers
Normative language	MUST, SHOULD, and MAY follow RFC-style meanings

1. Executive Summary

APGAR is a CAD-neutral autorouting library designed to turn GPU throughput into higher-quality PCB layouts. Its primary architectural choice is to separate exact PCB geometry from disposable GPU search representations. The GPU is used to generate diverse route candidates, negotiate board-wide resource allocation, and accelerate geometric analysis; exact geometry and the host CAD engine remain the final authorities for legality.

Core thesis

APGAR should optimize combinations of reusable route candidates across the whole board, not merely accelerate one shortest-path query at a time.

The first release will focus on single-ended nets, through vias, H/V/45-degree routing, a small set of rule buckets, incremental compilation, exact candidate validation, and a CPU reference pathfinder. Specialty routing for differential pairs, buses, BGA escape, length tuning, and signal-integrity-aware objectives is explicitly designed into interfaces but deferred.

1.1 Architecture at a glance

CAD Adapter -> Exact Board IR -> Geometry Compiler

- |-> Sparse tile atlas
- |-> Directional edge masks
- |-> Distance / margin fields
- |-> Topological portals
- |-> Via-template feasibility

Compiled Board + Routing Request

- > Candidate Generators
- > Candidate Store
- > Global Allocator / Negotiated Prices
- > Exact Legalizer + Incremental DRC
- > Host CAD Validation + Commit

1.2 Architectural priorities

1. Correctness and reproducibility before throughput.
2. Candidate diversity before raw candidate count.
3. Board-level optimization before per-net greediness.
4. Sparse and incremental representations before monolithic full-board graphs.
5. Exact geometric validation before CAD commit.
6. Benchmarkable subsystem boundaries before premature unification.

2. Scope and Non-Goals

2.1 Version 0.x scope

- A reusable C++ library with a CUDA-first compute backend.
- A neutral exact PCB Board IR with integer coordinates.
- Import/export adapters, initially targeting KiCad-compatible workflows.

- Conservative, tile-based GPU search-field compilation.
- CPU reference A*/Dijkstra and at least two GPU route generators.
- Multi-candidate storage, deduplication, scoring, and global selection.
- Exact route legalization and incremental DRC for supported primitives.
- Deterministic replay, checkpoints, telemetry, and benchmark tooling.

2.2 Deferred capabilities

- Full copper-pour repouring and plane synthesis.
- Native curved arbitrary-angle routing beyond line/arc legalization.
- Differential-pair, bus, BGA escape, and length-tuning production support.
- Electromagnetic field solving or sign-off signal-integrity analysis.
- Placement optimization.
- A standalone end-user PCB editor.

2.3 Explicit non-goals

- Claiming globally optimal routing for industrial boards.
- Treating a raster or lattice as the authoritative board representation.
- Depending on one GPU pathfinding algorithm for all geometries.
- Replacing the host CAD engine's final connectivity and DRC authority.
- Embedding KiCad-specific object semantics into the routing core.

3. Terminology and Normative Conventions

Term	Definition
Exact Board IR	Canonical CAD-neutral representation of physical geometry, rules, stack-up, nets, and locked copper.
Compiled Board	Disposable GPU-oriented representation derived from the Exact Board IR for a specific compiler profile.
Rule bucket	A group of nets or obstacles that share equivalent routing width and clearance behavior.
Resource	A capacity-bearing routing abstraction, such as a tile corridor, directional edge span, portal, or via site.
Candidate	A reusable possible route or route tree for one net, with geometry, resource footprint, and quality metrics.
World	One concurrent global-allocation state with its own prices, selections, and objective weights.
Guide	A selected candidate or corridor that may still require exact geometric legalization.
Legalization	Conversion or adjustment of a guide into exact CAD primitives satisfying supported rules.
Authoritative validation	Final validation by APGAR exact DRC and, before commit, the host CAD engine.

MUST identifies a mandatory requirement; SHOULD identifies a strong default that may be overridden with documented rationale; MAY identifies an optional capability.

4. System Requirements

4.1 Functional requirements

- FR-001** The core library MUST accept a CAD-neutral Exact Board IR without requiring a live editor process. *Rationale: Enables headless testing, cloud execution, and multiple CAD adapters.*
- FR-002** Every emitted route MUST be traceable to a routing request, configuration fingerprint, seed, compiler profile, and candidate provenance record. *Rationale: Required for reproducibility and debugging.*
- FR-003** The compiler MUST conservatively represent supported geometry: it MUST NOT mark an exactly illegal movement as legal. *Rationale: False-free search space is a correctness defect.*
- FR-004** The system MUST support partial boards containing locked and pre-routed copper. *Rationale: Interactive and incremental workflows are primary use cases.*
- FR-005** The router MUST support multiple candidate generators behind one interface. *Rationale: No single routing primitive dominates all PCB geometries.*
- FR-006** The allocator MUST be able to select candidates without rerunning geometric pathfinding. *Rationale: Candidate reuse is the central performance hypothesis.*
- FR-007** The legalizer MUST validate exact geometry before a route can be committed. *Rationale: Search fields are approximate.*
- FR-008** The session layer MUST support incremental invalidation and recompilation after localized edits. *Rationale: Full rebuilds would undermine interactive use.*
- FR-009** A CPU reference implementation MUST exist for every correctness-critical GPU primitive. *Rationale: Provides oracle behavior on small and medium tests.*
- FR-010** The library MUST expose machine-readable telemetry and benchmark results. *Rationale: Algorithm selection must be evidence-driven.*

4.2 Quality attributes

Attribute	Target / architectural response
Correctness	Exact integer geometry, conservative compilation, dual DRC validation, invariant checks.
Determinism	Seeded randomness, fixed-point costs where practical, stable tie-breaking, recorded hardware/backend.
Scalability	Sparse tile atlas, ROI batching, procedural adjacency, compressed candidate resources.
Incrementality	Generational object IDs, dirty tile halos, dependency tracking, persistent sessions.
Portability	Backend abstraction; CUDA-first implementation with clean isolation from core semantics.
Observability	Per-stage timing, kernel counters, memory accounting, candidate-diversity metrics, replay bundles.

Attribute	Target / architectural response
Extensibility	Versioned IR and plugin-like route-generator/specialty-router interfaces.

5. Architecture Decisions

ADR-001 - ACCEPTED: Exact geometry is authoritative

The Exact Board IR and exact checker define correctness. GPU rasters, fields, graphs, and guides are compiled caches and may be regenerated or replaced.

ADR-002 - ACCEPTED: Candidate generation is separated from global allocation

Route generators produce reusable candidates. The allocator chooses combinations under shared resource prices and may request additional candidates around hotspots.

ADR-003 - ACCEPTED: Regular adjacency is procedural

APGAR will not materialize a CSR edge for every regular grid neighbor. Neighbor identities and costs are derived arithmetically; sparse structures are reserved for portals and exceptional transitions.

ADR-004 - PROPOSED: C++ core with CUDA-first backend

The core will target modern C++ and a CUDA backend first. GPU APIs are isolated behind runtime interfaces so HIP or other backends can be evaluated without changing routing semantics.

ADR-005 - ACCEPTED: Integer global coordinates

Global board coordinates use signed 64-bit integer database units. GPU kernels use tile-local compact coordinates and explicit transforms.

ADR-006 - PROPOSED: Hybrid raster and topological routing

Dense directional fields handle constrained detailed search; sparse portals and free-space topology identify long-distance and homotopically distinct alternatives.

ADR-007 - ACCEPTED: Host CAD remains final authority

Before commit, generated primitives are checked by the host CAD engine. APGAR's own DRC enables speed and isolation but does not silently replace editor semantics.

6. System Context and Boundaries

6.1 External actors

Actor	Interaction
EDA editor	Supplies board state, routing request, locks, rules, and user intent; previews or commits results.
Headless CLI/service	Loads serialized Board IR, runs route sessions, exports solutions and replay artifacts.
Host CAD DRC/connectivity engine	Performs final validation and returns structured violations.
Benchmark harness	Runs deterministic corpora, gathers metrics, compares baselines and configurations.
GPU runtime/driver	Executes compiled kernels and reports device capabilities and failures.

6.2 Trust boundaries

- Imported CAD data is untrusted until normalized and validated.
- GPU outputs are untrusted until bounds, predecessor, and geometry invariants pass.
- Serialized sessions are versioned and checksummed.
- A host CAD rejection overrides APGAR's internal legality assessment.

7. Component Architecture

Module	Primary responsibility
board_ir	Canonical entities, IDs, units, rules, net constraints, geometry ownership.
geometry	Exact predicates, polygon operations, offsetting, distance and intersection queries.
geometry_compiler	Rule buckets, sparse tiles, masks, fields, portals, via feasibility, dependency graph.
gpu_runtime	Device discovery, allocators, streams, kernels, scans, queues, profiling, error containment.
route_generators	Sweep, frontier, topological, line-explore, multipin, and future specialty engines.
candidate_store	Candidate geometry, provenance, metrics, signatures, indices, persistence, pruning.
global_allocator	Resource capacities, prices, world states, selection, conflict analysis, column generation.
legalizer	Guide-to-geometry conversion, local shifts, jogs, corner styles,

Module	Primary responsibility
	via relocation, shove hooks.
drc	Incremental broad phase, exact narrow phase, supported-rule checking, violation explanations.
session	Incremental state, dirty propagation, checkpoints, cancellation, deterministic replay.
cad_adapters	Import normalization, preview, host validation, transaction-safe commit.
bench	Synthetic generators, corpus manifests, metric collection, statistical comparison.

7.1 Dependency rules

- board_ir MUST NOT depend on a CAD adapter or GPU backend.
- geometry_compiler MAY depend on geometry and board_ir, but MUST NOT depend on a specific editor.
- route_generators MUST consume versioned compiled views, not mutable CAD objects.
- global_allocator MUST consume candidate/resource abstractions, not exact editor primitives.
- legalizer and drc MAY access exact geometry but MUST return changes as transactions.
- cad_adapters MUST translate rather than leak editor object identity into the core.

8. Exact Board IR

8.1 Identity and versioning

Every persistent entity uses a stable 64-bit ID scoped to one board document. Mutations increment an entity generation. References contain both ID and expected generation where stale use would be dangerous. Serialized Board IR includes a schema version, unit scale, content hash, and adapter metadata.

```
using DbCoord = int64_t;
using EntityId = uint64_t;
using Generation = uint32_t;

struct EntityRef {
    EntityId id;
    Generation generation;
};

struct Point64 {
    DbCoord x;
    DbCoord y;
};
```

8.2 Core entities

Entity	Required fields
Board	outline/cutouts, stack-up, global rules, object tables, revision, coordinate units
Layer	physical order, type, thickness, material metadata, routability flags
Net	terminal references, net class, constraints, priority, criticality
Terminal	padstack geometry, allowed connection regions, component and pin identity
Track	net, layer, width profile, line/arc primitives, lock state
Via	net, via template, position, start/end layer, lock state
Obstacle	layer geometry, class, ownership, keepout and interaction flags
RuleSet	clearance matrix, width ranges, via rules, neck-down permissions, region overrides
ConstraintGroup	differential, bus, length, topology, layer, return-path metadata
Region	polygonal scope and rule/behavior overrides

8.3 Geometry primitives

- Integer points and vectors.
- Closed polygon sets with holes and normalized winding.
- Finite line segments with width or width profile.
- Circular arcs represented by center/radius/start/sweep or equivalent exact-safe form.
- Circles, capsules, rounded rectangles, slots, and polygonal pad shapes.
- Layer-aware pad stacks and via pad stacks.

IR-001 Geometry normalization MUST reject self-intersecting outlines unless the adapter resolves them into valid polygon sets.

IR-002 All imported floating-point coordinates MUST be quantized once at the adapter boundary and never repeatedly converted.

IR-003 Every obstacle MUST retain enough provenance to explain a collision to the user.

8.4 Rules and rule buckets

The Exact Board IR preserves the full rule system. The compiler derives rule buckets for search efficiency. Bucket equivalence is defined by identical movement width, relevant pairwise clearances, allowed layers, and via-template eligibility for the current compilation mode.

```
struct RoutingProfile {  
    DbCoord nominal_width;  
    DbCoord minimum_width;  
    RuleClassId moving_class;  
}
```



```

LayerMask allowed_layers;
ViaTemplateMask allowed_vias;
HeadingMask allowed_headings;
};

struct RuleBucketKey {
    RoutingProfile profile;
    ObstacleInteractionSignature interactions;
};

```

9. Geometry Compiler

9.1 Compiler contract

The compiler transforms an immutable Board IR snapshot into one or more immutable Compiled Board views. A view is parameterized by grid profile, heading set, rule-bucket set, tile dimensions, and compiler version.

GC-001 A compiled legal edge **MUST** imply that its swept exact trace envelope is legal against all represented static obstacles.

GC-002 Compiler conservatism **MUST** be measurable by comparing compiled reachability with exact local search on generated microcases.

GC-003 Compiled artifacts **MUST** carry the Board IR content hash and compiler-profile fingerprint.

9.2 Multi-resolution sparse tile atlas

APGAR uses a coarse full-board capacity map plus sparse detailed tiles. Fine tiles are instantiated around terminals, obstacles, narrow channels, selected topological corridors, and congestion hotspots. Each tile includes a halo large enough to evaluate the largest supported swept envelope relevant to that tile.

Tile field	Purpose
tile descriptor	origin, scale, layer, rule bucket, generation, halo, residency
directional legality	bit masks for H/V/diagonal movements and optional turn transitions
static cost	length, layer preference, margin, region penalties
dynamic price	present and historical congestion prices
distance/margin	clearance guidance and channel-center preference
terminal seeds	source/destination connection states
portal links	sparse links to adjacent resolution levels or topological structures
via feasibility	legal via-template sites and span metadata

9.3 Directional edge legality

Node occupancy alone is insufficient. For every supported heading, the compiler evaluates the swept capsule or exact envelope between adjacent samples. Diagonal corner cutting is prohibited unless the entire diagonal movement envelope is legal.

Conservatism rule

False-blocked space may reduce route quality and must be measured. False-free space is a correctness defect and must fail tests.

9.4 Distance and margin fields

Distance fields guide routing toward corridor centers and estimate whether a region can accommodate a route profile. They are advisory rather than authoritative because they inherit rasterization error. Exact Euclidean distance transforms are GPU-suitable; the Parallel Banding Algorithm is a relevant implementation reference [R6].

9.5 Topological portals

- Obstacle-corner and tangent portals.
- Tile-boundary gates with capacity estimates.
- Pad/escape landing portals.
- Via-site and layer-transition portals.
- Navigation-mesh or decomposed free-space adjacency.
- User or adapter supplied preferred corridors.

Topological routes define broad homotopy choices and detailed regions of interest. They do not certify exact route legality.

9.6 Via-template compilation

```
struct ViaTemplate {
    ViaTemplateId id;
    LayerId start_layer;
    LayerId end_layer;
    DbCoord drill;
    SmallVector<LayerShape, 16> conductive_shapes;
    SmallVector<LayerShape, 16> keepout_shapes;
    uint32_t process_flags;
    CostVector base_cost;
};
```

GC-010 Via feasibility MUST be evaluated using complete padstack and keepout geometry across every affected layer.

GC-011 Full all-pairs layer transitions MUST NOT be materialized as regular graph edges.

GC-012 Sparse legal via sites MAY be generated from clearance maxima, portals, pad escapes, and user constraints.

9.7 Incremental invalidation

Every compiled tile records dependencies on exact entities and neighboring halo tiles. Mutations generate dirty bounding regions per layer and interaction class. Dirty regions expand by the maximum affected clearance and invalidate only overlapping artifacts.

```
BoardDelta
-> normalize affected entity bounds
-> expand by rule-dependent influence radius
-> mark exact spatial-index nodes dirty
-> mark compiled tile interiors + halos dirty
-> invalidate dependent portals and via sites
-> rebuild asynchronously
-> atomically publish new compiled generation
```

10. GPU Runtime and Execution Model

10.1 Runtime responsibilities

- Capability discovery and backend selection.
- Stream and event management.
- Long-lived device memory pools.
- Tile residency and upload scheduling.
- Prefix scan, segmented reduction, sort, compaction, and queue primitives.
- Kernel launch tracing and deterministic error capture.
- Cancellation at safe batch boundaries.

10.2 Memory tiers

Tier	Content	Lifetime
Host exact	Board IR, exact geometry, spatial indices	board/session
Host compiled cache	compressed tiles, portals, serialization buffers	session
Device persistent	resident tiles, resource prices, candidate indices	session
Device batch	distance labels, frontiers, predecessors, temporary reductions	route batch
Device world	selection arrays, world prices/deltas, objective accumulators	allocator run

GPU-001 The runtime MUST report peak and current memory by subsystem.

GPU-002 Out-of-memory conditions MUST be recoverable through smaller batches, tile eviction, or CPU fallback.

GPU-003 Kernel errors MUST invalidate the current result and produce a replayable diagnostic bundle.

10.3 Deterministic computation

- Integer or fixed-point primary routing costs where practical.
- Stable composite keys for atomic minimum operations.
- Deterministic compaction and stable ordering at externally visible boundaries.
- Counter-based random number generation keyed by board hash, net ID, candidate ordinal, and world ID.
- Hardware/backend identity recorded with results.

Determinism level

Bit-identical results are required on the same backend and supported device class. Cross-backend semantic equivalence is required; bit identity across CUDA and future backends is not initially required.

11. Common Routing State and Cost Model

11.1 Search state

The generic detailed state includes physical layer, tile/cell coordinate, incoming heading, and optional finite state for special constraints. State extensions must be bounded and explicitly declared so memory use can be estimated.

```
struct RouteState {
    LayerId layer;
    TileId tile;
    LocalCoord x;
    LocalCoord y;
    Heading heading;
    uint16_t extension_state; // generator-specific finite state
};
```

11.2 Cost vector

APGAR preserves a quality vector on candidates and uses scalarized fixed-point costs inside individual searches. Search scalarization is configuration-controlled and recorded. Hard illegality is never represented as a large finite cost.

Metric	Examples
Geometric	length, bend count, corner style, margin
Layer/via	layer preference, via count, span, microvia stack cost
Congestion	present price, historical price, scarce portal use
Electrical proxy	reference discontinuity, coupled length, stub penalty
Manufacturing	neck-down, acute-angle risk, drill family, testability proxy
Constraint error	length error, skew, ordering or topology deviation

11.3 Resource model

Resources are capacity-bearing abstractions used by the allocator. They are intentionally coarser than exact geometry, but candidate footprints may include fine edge spans where needed.

- Directional tile-edge spans.
- Coarse corridor bins.
- Topological portals.
- Via sites and via-stack templates.
- Pad escape channels.
- Tuning-zone capacity.
- Special shared constraints such as bus lanes.

RES-001 Every candidate MUST provide a sorted, canonical resource footprint.

RES-002 Resource capacity and usage units MUST be explicit and compatible.

RES-003 Exact geometric collisions discovered after allocation MUST be projectable back to one or more resources or trigger resource refinement.

12. Candidate Generator Interface

```
class ICandidateGenerator {
public:
    virtual GeneratorCapabilities capabilities() const = 0;

    virtual CandidateBatch generate(
        const CompiledBoardView& board,
        const NetRouteRequest& request,
        const PriceSnapshot& prices,
        const CandidateGenerationPolicy& policy,
        CancellationToken cancel) = 0;
};
```

12.1 Generator requirements

GEN-001 A generator MUST declare supported headings, via models, terminal types, and multipin behavior.

GEN-002 A generator MUST return failure explanations distinguishing unreachable, resource-exhausted, unsupported, and cancelled.

GEN-003 Every candidate MUST include provenance sufficient to reproduce it.

GEN-004 Generators SHOULD support banned or penalized resource sets to produce diverse alternatives.

GEN-005 Generators MUST NOT mutate global resource usage while searching.

12.2 Heading-aware sweep router

The sweep router generalizes GAMER's scan-based shortest-path propagation to PCB heading states. GAMER demonstrates that rectilinear multisource-multidestination routing can be transformed into GPU-friendly prefix operations and achieved large maze-routing kernel speedups in VLSI contexts [R1]. APGAR will test horizontal, vertical, and diagonal segmented scans, followed by local heading-turn and via relaxations.

```
repeat until convergence, destination bound, or round budget:
    segmented_scan(E/W states)
```

```

segmented_scan(N/S states)
segmented_scan(NE/SW states)
segmented_scan(NW/SE states)
relax_same_cell_heading_changes()
relax_sparse_via_transitions()
update_destination_bounds()
compact_changed_regions()

```

- Best expected case: dense regular ROIs with long runs and moderate bend counts.
- Risk: many short obstacle-fragmented runs and highly tortuous paths may require excessive rounds.
- Prototype must compare exactness, path-cost gap, and examined-state work against CPU reference.

12.3 Bucketed frontier router

A frontier-based GPU router provides a complementary strategy for sparse exploration. Candidate algorithms include delta-stepping, integer-cost buckets, or multi-queue variants. The first implementation should prioritize deterministic behavior and bounded memory over theoretical generality.

- Best expected case: sparse reachable regions or strongly directed heuristics.
- Use compact active-state queues and duplicate-tolerant relaxation.
- Support ROI expansion when the initial corridor proves insufficient.

12.4 Topology-first router

The topology-first router enumerates portal paths representing distinct obstacle bypass choices, then invokes a detailed generator inside each corridor. InstantGR's DAG-based representation demonstrates how route alternatives and their resource footprints can expose more parallelism than bounding-box conflict tests [R2].

12.5 Adaptive line-exploration router

A gridless or semi-gridless generator should derive exploration points from obstacles, rules, and destination direction. The 3D LineExplore work is relevant evidence that continuous-space exploration can avoid fixed-grid resolution constraints and can natively incorporate layer transitions [R5]. APGAR should treat this as a candidate generator rather than adopting its one-pass routing paradigm as the whole-board optimizer.

12.6 Generator dispatcher

Feature	Dispatcher signal
ROI occupancy density	Dense fields favor scans; sparse fields favor frontier/topological methods.
Run fragmentation	Short segmented runs reduce sweep efficiency.
Expected turn count	High turn complexity may favor frontier or topology-first search.
Portal dominance	Sparse portal graph favors topology-first.
Batch regularity	Similar ROI sizes and rule buckets favor GPU batching.
Candidate diversity deficit	Choose a generator with a different topology bias.

DSP-001 Dispatcher decisions **MUST** be logged with features and selected policy.

DSP-002 The benchmark harness **MUST** support forcing any compatible generator to avoid self-confirming dispatch heuristics.

13. Candidate Representation and Store

```
struct RouteCandidate {
    CandidateId id;
    NetId net;
    GeneratorId generator;
    Provenance provenance;

    CompressedRouteGeometry geometry;
    CompressedResourceFootprint resources;
    CostVector metrics;
    ConstraintAssessment constraints;
    Hash128 resource_signature;
    Hash128 geometry_signature;
};
```

13.1 Geometry representation

- Ordered line/arc/via primitives for two-terminal routes.
- Rooted branch representation for multipin route trees.
- Optional corridor envelopes and movable degrees of freedom for legalization.
- Canonical simplification of collinear segments and redundant waypoints.
- Exact integer endpoints after candidate validation.

13.2 Resource compression

- Run-length encoded directional edge spans.
- Sorted portal and via-site IDs.
- Coarse resource bitmap for fast overlap screening.
- Fine sparse list for exact allocator accounting.
- Optional per-resource fractional usage for capacity models wider than one track.

13.3 Diversity and deduplication

Candidate count is not a useful metric without diversity. APGAR maintains exact duplicate hashes and approximate similarity statistics. A candidate may be rejected when it adds no new resource footprint, topology, or useful Pareto tradeoff.

Diversity metric	Use
Resource Jaccard overlap	Detect candidates competing for the same board resources.
Geometric overlap ratio	Detect nearly coincident paths with minor waypoint noise.
Portal sequence	Approximate homotopy/topological distinction.
Layer/via signature	Preserve meaningful layer and transition alternatives.

Diversity metric	Use
Metric dominance	Prune candidates worse on every objective with no unique feasibility benefit.

CAN-001 Candidate stores MUST enforce per-net memory budgets.

CAN-002 Pruning MUST preserve candidates currently selected by retained world states.

CAN-003 A candidate rejected by exact DRC MUST retain a structured failure record to improve later generation.

14. Global Allocator

14.1 Problem definition

For each routable net, the allocator selects one candidate while respecting resource capacities and minimizing a configurable board-level objective. Intermediate selections may overuse resources; feasibility is approached through negotiated prices, mirroring the useful Pathfinder principle that temporarily infeasible shared-resource states can help explore the solution space [R3].

```
score(net, candidate, world) =
  intrinsic_quality(candidate, world.weights)
+ sum(resource_price[world, r] * candidate.usage[r])
+ constraint_penalties(candidate, net, world)
```

14.2 Iterative selection

7. Select the minimum-score candidate for every net in each world.
8. Accumulate resource usage in parallel.
9. Measure over-capacity resources and exact-risk indicators.
10. Update present and historical prices.
11. Identify nets whose candidate sets are insufficient around expensive resources.
12. Request targeted new candidates under the current price snapshot.
13. Prune dominated candidates and continue until feasible, stalled, or budget exhausted.

14.3 Multi-world execution

Multiple worlds share the immutable candidate pool and base resource model but maintain independent selections, price states, objective weights, perturbations, and update schedules. This allows APGAR to spend additional GPU capacity on board-level search rather than storing full independent pathfinding state for every world.

ALL-001 World state MUST be compact enough to run tens or hundreds of worlds for moderate candidate pools.

ALL-002 The allocator MUST retain a Pareto set of feasible or near-feasible worlds.

ALL-003 Price updates MUST be bounded and recorded to prevent silent numerical instability.

14.4 Column generation

Candidate generation acts as a pricing subproblem: nets touching high-price resources are rerouted against the latest price field to discover alternatives with lower reduced cost. The allocator is therefore not limited to its initial candidate pool.

- Hotset based on overused resources and affected nets.
- Resource bans or strong penalties to force topological novelty.
- Per-net candidate generation budget based on criticality and conflict impact.
- Stall detection when new candidates fail to improve diversity or reduced cost.

14.5 Parallel conflict safety

Generators read immutable price snapshots and do not directly commit global occupancy. Selection and usage accumulation occur in explicit phases, avoiding races caused by many nets mutating a shared congestion map during search. OrthoRoute is useful direct PCB prior art but routes nets sequentially on a shared congestion map while parallelizing each net's SSSP; APGAR's candidate/allocation split is intended to expose additional net-level and world-level parallelism [R4].

15. Multipin Nets

Multipin nets are represented as route trees. Fixed decomposition into independently optimized two-pin connections is not the canonical representation because it can discard useful topology.

14. Choose one terminal or existing branch set as the initial tree.
15. Run multisource-to-multidestination search from the tree to unconnected terminals.
16. Attach one or more terminals according to the generator policy.
17. Repeat under alternate root, order, Steiner, layer, and price perturbations.
18. Canonicalize the final tree and compute one resource footprint.

MP-001 A multipin candidate **MUST** maintain connectivity under branch simplification.

MP-002 Candidate comparison **MUST** account for shared trunk resources only once.

MP-003 Partial repair **MAY** replace a branch without regenerating the entire tree.

16. Exact Legalization

16.1 Legalizer contract

The legalizer converts a selected candidate guide into exact CAD primitives or returns structured conflicts. It operates transactionally: proposed edits are validated before replacing existing board state.

- Snap or translate segments within an allowed corridor.
- Move a via among feasible sites.
- Insert local jogs.
- Replace sharp corners with configured miters or arcs.
- Apply legal neck-downs near terminals.
- Invoke bounded shove operations on explicitly movable copper.
- Request candidate substitution or local regeneration.

LEG-001 Legalization **MUST NOT** silently change a net's hard constraints.

LEG-002 Every adjustment **MUST** remain within declared candidate degrees of freedom or trigger candidate re-evaluation.

LEG-003 Legalization failure MUST identify conflicting primitives, violated rules, and suggested repair scope.

16.2 Transaction model

```
begin RouteTransaction
  stage removals / replacements
  stage new exact primitives
  run APGAR incremental DRC
  run host CAD validation (adapter)
  if both pass:
    commit atomically
  else:
    rollback and retain diagnostics
end
```

17. DRC and Geometric Verification

17.1 Two-level checking

Level	Role
Compiled legality	Fast conservative rejection during search.
Exact APGAR DRC	Incremental validation and diagnostic generation for supported rules.
Host CAD validation	Authoritative final editor-specific connectivity and rule validation.

17.2 Spatial acceleration

Exact checking uses per-layer spatial bins and hierarchical bounding volumes for broad-phase candidate generation, followed by exact or conservatively bounded narrow-phase predicates. OpenDRC provides relevant evidence for layer-wise BVHs, adaptive partitioning, and edge-based GPU kernels in physical verification [R7].

- Dirty-region broad phase.
- Batched segment-segment, point-segment, arc, pad, and via checks.
- Pairwise rule lookup using moving and obstacle classes.
- Connectivity checks at intended terminal and branch junctions.
- Violation objects containing exact witnesses and provenance.

DRC-001 Supported exact rules MUST have generated property tests around equality and one-unit boundary cases.

DRC-002 Unsupported rules MUST be declared before routing and delegated to the host, not assumed legal.

DRC-003 The internal checker and host checker disagreement MUST be retained as a regression artifact.

18. Incremental Session Architecture

18.1 Persistent session state

- Board IR snapshot and revision lineage.
- Exact spatial indices.
- Compiled tile generations and residency.
- Candidate pools and invalidation dependencies.
- Resource history and retained allocator worlds.
- Host adapter mapping and pending transactions.
- Reproducibility configuration and telemetry.

18.2 Delta classes

Delta	Likely response
Component/pad moved	Invalidate terminal geometry, nearby tiles/portals, affected net candidates, exact spatial index.
Rule changed	Rebucket affected nets/obstacles; invalidate dependent compiled views and candidates.
Track locked/unlocked	Update obstacle/movable classification and local resource capacities.
Layer stack changed	Invalidate stack-dependent via templates, all compiled layer transforms, and affected candidates.
Net constraint changed	Invalidate that net's candidate metrics or geometry depending on constraint type.
Local manual route edit	Treat edited copper as locked or movable according to user action; reroute impacted hotset.

INC-001 An incremental update **MUST** produce the same legal result set as a clean rebuild for the same deterministic policy, modulo explicitly allowed cache-sensitive candidate ordering.

INC-002 The session **MUST** expose why each artifact was invalidated.

19. Specialty Routing Extension Points

19.1 Differential pairs

Differential pairs should be modeled as coupled ribbon states rather than two independent traces. The future extension state includes pair gap, side order, skew/phase, legal coupled-via template, and uncoupled-length budget.

19.2 Buses

Bus candidates contain a shared corridor, lane order, legal permutation points, synchronized transitions, and aggregate tuning capacity.

19.3 Escape routing

Escape routing generates multiple legal pad-to-portal motifs and allocates channels before area routing. Portal outputs become normal terminals for later candidate generators.

19.4 Length tuning

Candidates carry current length, required added length, available tuning-zone capacity, and interference footprint. Global allocation reserves tuning space rather than deferring all feasibility to post-processing.

19.5 Return-path awareness

A future route profile may identify a reference conductor and penalize plane-split crossings, layer transitions lacking return-via options, and excessive discontinuity.

20. Public API and Data Contracts

20.1 Session API sketch

```
class ApgarSession {
public:
    static Result<ApgarSession> create(BoardSnapshot, SessionConfig);

    Result<DeltaReceipt> apply_delta(BoardDelta);
    Result<CompileReceipt> compile(CompileRequest);
    Result<RouteJob> route(RouteRequest);
    Result<Preview> preview(const SolutionId&);
    Result<ValidationReport> validate(const SolutionId&);
    Result<CommitPlan> prepare_commit(const SolutionId&);
    Result<ReplayBundle> export_replay(const JobId&);
};
```

20.2 Route request

Field	Meaning
net selection	all eligible nets, explicit set, region, or conflict hotset
objective profile	weights, lexicographic priorities, and hard bounds
generator policy	allowed/forced generators, candidate and time budgets
allocator policy	world count, iterations, price schedule, stall thresholds
edit policy	locked/movable existing copper and permitted shove scope
determinism	seed and requested determinism level
resource limits	VRAM, host memory, wall-clock/cancellation budgets

20.3 Error model

- InvalidInput: malformed Board IR or unsupported semantic combination.
- Unsupported: well-formed request outside current capability.
- Unreachable: no path under the exact supported model and current locks.
- ResourceExhausted: memory or bounded search budget exhausted.
- ValidationFailed: candidate or solution violates exact or host rules.
- BackendFailure: device, driver, or kernel failure.
- Cancelled: cooperative cancellation.
- InternalInvariant: logic defect requiring replay artifact.

21. Serialization, Checkpoints, and Replay

- Versioned Board IR snapshot or reference plus content hash.
- Normalized routing configuration.
- Compiler profile and generated-artifact manifests.
- Candidate provenance and selected solution.
- Allocator price/update history or compact replay seed.
- Backend/device metadata and build IDs.
- Failure diagnostics and exact/host DRC reports.

SER-001 Serialized formats MUST use explicit schema versions and reject incompatible major versions.

SER-002 Replay bundles MUST omit proprietary board content only when configured; redaction must be explicit.

SER-003 Checkpoint restoration MUST verify hashes before reusing compiled artifacts.

22. CAD Adapter Contract

22.1 Adapter responsibilities

19. Read editor-native board state into normalized Board IR.
20. Map editor rules into core semantics and declare unsupported rules.
21. Preserve stable mapping between core entities and editor objects where possible.
22. Provide preview overlays without mutating the board.
23. Validate proposed transactions through the editor engine.
24. Commit atomically or provide rollback behavior.

CAD-001 Adapters MUST NOT approximate unsupported host rules as weaker core rules.

CAD-002 A host validation rejection MUST include native violation data when available.

CAD-003 Adapter integration tests MUST include round-trip coordinate and layer mapping.

23. Concurrency, Cancellation, and Failure Recovery

- Immutable snapshots are published using generation IDs.
- Compilation and candidate generation may run concurrently against the same snapshot.
- Board mutations create a new revision rather than modifying data visible to a running job.
- Jobs are cancelled at bounded batch boundaries; long kernels require split launch design.
- A failed device job never partially commits board geometry.
- CPU fallback may continue from persistent candidates and exact state when semantics match.

24. Security and Robustness

- Validate all array dimensions and multiplication overflow before allocation.
- Treat imported counts and polygon complexity as adversarial.
- Bound candidate counts, path lengths, and recursion.
- Use checksummed serialization and reject malformed indices.
- Do not execute code or load arbitrary GPU binaries from board files.
- Separate diagnostic board artifacts from telemetry opt-in.

25. Verification and Testing Strategy

25.1 Test pyramid

Level	Examples
Unit	geometry predicates, rule lookup, coordinate transforms, resource accounting
Property	conservatism, symmetry, monotonic clearance, boundary equality, random microboards
Differential	CPU versus GPU path costs, predecessor reconstruction, DRC agreement
Integration	compile-route-legalize-validate, incremental delta equivalence
Corpus	synthetic families and real open-source boards
Performance	kernel throughput, candidate/sec, world iterations/sec, memory scaling
Fault	OOM, cancellation, corrupted checkpoint, GPU failure, host rejection

25.2 Correctness invariants

- Every candidate forms one connected route or tree over exactly the intended terminals.
- No predecessor chain cycles or exits allocated state.
- Compiled legal movement is exact-legal against represented static geometry.
- Resource usage accumulation equals a CPU recomputation.
- A committed solution has zero supported APGAR DRC violations and zero host-blocking violations.
- Incremental and clean rebuilds agree on normalized exact geometry and supported legality.

25.3 Microcase generator

A generated microcase framework should create small exact boards with controlled obstacle separations, corners, slots, pad shapes, layer transitions, and one-unit boundary perturbations. Many cases can be solved exhaustively to provide optimality and reachability oracles.

26. Benchmark and Evaluation Plan

26.1 Metrics

Category	Metrics
Feasibility	connected nets, unrouted terminals, APGAR DRC violations, host DRC violations
Quality	length, vias/span, bends, minimum margin, layer use, constraint error
Exploration	candidates/net, unique resource signatures, overlap, topology diversity, best-of-k curve
Allocator	overused resources, price convergence, worlds retained, columns generated
Performance	end-to-end time, stage time, candidates/sec, world iterations/sec, latency
Memory	persistent, batch, candidate, world, exact-index, peak VRAM
Incremental	dirty area, artifacts rebuilt, reroute latency, solution churn

26.2 Benchmark tiers

25. Small exact instances solvable by exhaustive search or mathematical optimization.
26. Controlled synthetic families: channels, pin fields, crossbars, mazes, via bottlenecks, topology alternatives.
27. Real open-source boards validated by a CAD engine.

PCBWorld is a newly released engine-grounded KiCad benchmark environment with synthetic families and 679 real open-source boards, and is a promising external evaluation path once its tooling is integrated [R8]. Results from unrelated PCB papers must not be compared without normalizing geometry, design rules, allowed routing behavior, and completion definitions.

26.3 Initial experiments

28. CPU A* versus GPU sweep versus GPU frontier on identical compiled fields.
29. Uniform full-grid baseline versus sparse tiled compilation.
30. Sequential negotiated routing versus candidate allocation at equal wall-clock budgets.
31. Candidate pools of 4, 8, 16, 32, 64, and 128 candidates per net.
32. One world versus multi-world search at equal memory and time budgets.
33. Incremental component movement versus clean rebuild.

27. Performance and Memory Budgets

Budgets are initial engineering targets, not promises. They are intended to prevent designs that only work on datacenter GPUs.

Scenario	Initial target
----------	----------------

Developer GPU	Useful operation on 12-16 GB VRAM for moderate boards.
Workstation GPU	Scale to 24-48 GB without changing algorithms.
Interactive local edit	Sub-second compilation for small dirty regions; route latency measured separately by scope.
Persistent compiled data	Dominated by occupied/detailed tiles rather than full board area.
Candidate store	Configurable hard cap by bytes and candidates/net.
Batch state	Automatically sized to leave recovery headroom.

OrthoRoute's documented full-lattice estimates illustrate why APGAR should avoid explicit high-dimensional full-board graph storage; its 200 mm x 200 mm, 32-layer example estimates eight million nodes and tens of gigabytes of VRAM [R4].

28. Observability and Diagnostics

- Structured event log with board/job/revision IDs.
- Stage and kernel timing with device utilization context.
- Memory watermark timeline.
- Per-net candidate generation outcomes.
- Allocator convergence traces and top conflicted resources.
- Exact and host DRC disagreement report.
- Visualizable tile, resource, price, candidate, and predecessor dumps.
- One-command replay bundle generation for invariant failures.

29. Implementation Roadmap

Phase	Deliverable	Exit criteria
0 - Foundations	Board IR, exact geometry, adapter fixture, build/test infrastructure	Round-trip and geometry property tests pass.
1 - Compiled fields	Sparse tiles, directional masks, rule buckets, CPU reference router	Conservatism suite passes; memory telemetry exists.
2 - GPU kernel bakeoff	Sweep and frontier prototypes	Differential correctness; dispatch benchmark report.
3 - Candidate store	Compression, dedup, metrics, provenance	Best-of-k and diversity curves reproducible.
4 - Global allocator	Prices, worlds, selection, column generation	Beats sequential baseline on selected synthetic families.
5 - Legalizer/DRC	Exact transaction pipeline and host validation	Zero supported violations on committed corpus outputs.

Phase	Deliverable	Exit criteria
6 - Incremental session	Deltas, dirty dependency tracking, checkpoints	Incremental equivalence and latency targets met.
7 - Specialty pilots	Escape and differential-pair prototypes	Interfaces validated without core redesign.

29.1 First vertical slice

Milestone M1

Import a small board into Exact Board IR; compile conservative sparse H/V/45-degree fields; generate multiple candidates with CPU A* and one GPU kernel; validate every candidate exactly; emit a reproducible benchmark and replay bundle.

- Two or four routable signal layers.
- Single-ended two-terminal nets.
- Through vias only.
- Line-segment output with configured mitered corners.
- No shove, copper pours, differential pairs, buses, or length tuning.

30. Risks and Mitigations

Risk	Mitigation
Raster conservatism blocks important routes	Measure false-blocked space; refine tiles/corridors; add topology/gridless generators.
Exact legalization dominates runtime	Build incremental broad phase early; batch exact checks; preserve movable guide degrees of freedom.
Candidate pool lacks diversity	Resource/topology diversity metrics; forced bans; multiple generators; targeted column generation.
Allocator oscillates or stalls	Historical pricing, multi-world schedules, stall diagnostics, candidate insufficiency detection.
GPU memory grows with board dimensions	Sparse tiles, procedural adjacency, ROI batching, eviction, CPU fallback.
Cost model produces ugly legal routes	Pareto metrics, user objective profiles, explainability, quality corpus reviews.
CAD semantics disagree	Host validation is authoritative; retain disagreement regressions; adapter conformance suites.
Specialty routing forces redesign	Finite-state extension and specialty candidate interfaces are specified before implementation.

31. Open Architecture Questions

ID	Question	Proposed experiment / decision trigger
OQ-01	What tile sizes and resolution hierarchy minimize total work?	Sweep board families across occupancy and feature scales.
OQ-02	How many heading states are needed in the first production mode?	Compare H/V, octilinear, and legalizer-assisted variants.
OQ-03	Can heading-aware sweeps converge efficiently on PCB-like tortuous paths?	Kernel bakeoff with turn-count-controlled mazes.
OQ-04	What resource granularity best predicts exact conflicts?	Correlate allocator overuse with legalization failures.
OQ-05	What candidate similarity metric best predicts global value?	Ablate pruning metrics and measure final quality.
OQ-06	How many multi-world states provide useful marginal quality?	Quality-versus-world-count curves under fixed budgets.
OQ-07	How should movable existing copper be represented for shove?	Prototype transaction and dependency semantics before geometry optimization.
OQ-08	Which internal DRC rules must be production-complete before adoption?	Prioritize by host rejection frequency and runtime contribution.

32. Engineering Governance

- Architecture decisions are recorded as ADRs with status, rationale, alternatives, and consequences.
- Every performance claim includes a corpus, configuration, hardware, and exact commit ID.
- Every new GPU optimization requires differential tests against a reference path.
- Every serialized schema change has compatibility tests.
- Every new route-quality objective includes an explainable metric and regression visualization.
- No unsupported design rule may silently degrade to a weaker interpretation.

33. Research-to-Architecture Traceability

Source	Adopted insight	APGAR adaptation
GAMER [R1]	GPU-friendly directional sweeps and prefix-style propagation	Heading-aware H/V/diagonal candidate generator; benchmarked rather than universal.
InstantGR [R2]	DAG route alternatives, resource-aware parallel batching, flexible layer transitions	Candidate/topology representation and footprint-aware parallel allocation.
PathFinder [R3]	Negotiated present and historical	Candidate-level resource prices and

Source	Adopted insight	APGAR adaptation
	congestion; temporarily infeasible states	multi-world selection.
OrthoRoute [R4]	Direct GPU PCB routing viability and practical full-lattice memory pressure	Avoid full explicit graph; expose net/world parallelism beyond per-net SSSP.
3D LineExplore [R5]	Adaptive continuous-space exploration and multilayer routing	Optional adaptive line-exploration generator and topological portals.
PBA [R6]	Exact GPU Euclidean distance transform	Advisory distance/margin fields.
OpenDRC [R7]	Layer hierarchy, adaptive partition, edge-based GPU checks	Incremental broad phase and batched exact-rule kernels.
PCBWorld [R8]	Engine-grounded evaluation on synthetic and real KiCad boards	External benchmark and host-verified metric strategy.

34. References

- [R1] S. Lin, J. Liu, E. F. Y. Young, and M. D. F. Wong, “GAMER: GPU-Accelerated Maze Routing,” IEEE TCAD, vol. 42, no. 2, pp. 583-593, 2023. DOI: 10.1109/TCAD.2022.3184281. <https://doi.org/10.1109/TCAD.2022.3184281>
- [R2] L. Xiao, S. Lin, J. Liu, Q. Duan, T.-Y. Ho, and E. F. Y. Young, “InstantGR: Scalable GPU Parallelization for 3-D Global Routing,” IEEE TCAD, vol. 45, no. 1, pp. 441-452, 2026. DOI: 10.1109/TCAD.2025.3573685. <https://doi.org/10.1109/TCAD.2025.3573685>
- [R3] L. McMurchie and C. Ebeling, “PathFinder: A Negotiation-Based Performance-Driven Router for FPGAs,” in Reconfigurable Computing, pp. 365-381. DOI: 10.1016/B978-012370522-8.50024-8. <https://doi.org/10.1016/B978-012370522-8.50024-8>
- [R4] B. Benchoff, “OrthoRoute: A GPU-accelerated PCB autorouter for KiCad,” source repository and project documentation. <https://github.com/bbenchoff/OrthoRoute>
- [R5] N. Sun et al., “3D LineExplore: a 3D line exploration method for multi-layer PCB geometric routing,” Scientific Reports, vol. 16, article 6588, 2026. DOI: 10.1038/s41598-026-36925-0. <https://doi.org/10.1038/s41598-026-36925-0>
- [R6] T.-T. Cao, K. Tang, A. Mohamed, and T.-S. Tan, “Parallel Banding Algorithm to Compute Exact Distance Transform with the GPU,” SI3D 2010. DOI: 10.1145/1730804.1730818. <https://doi.org/10.1145/1730804.1730818>
- [R7] Z. He, Y. Zuo, J. Jiang, H. Zheng, Y. Ma, and B. Yu, “OpenDRC: An Efficient Open-Source Design Rule Checking Engine with Hierarchical GPU Acceleration,” DAC 2023. <https://github.com/opendrc/opendrc>
- [R8] H. Song et al., “PCBWorld: A Benchmark Environment for Engine-Grounded PCB Design Automation,” arXiv:2607.05915, 2026. <https://arxiv.org/abs/2607.05915>
- [R9] CUHK-EDA XPlace GPU global routing implementation, including CUDA maze-routing sweep kernels. <https://github.com/cuhk-eda/Xplace>

Appendix A. Proposed Repository Layout

```
apgar/  
cmake/  
docs/  
  architecture/  
  adr/  
  research/  
include/apgar/  
  board_ir/  
  geometry/  
  compiler/  
  routing/  
  candidates/  
  allocator/  
  legalizer/  
  drc/  
  session/  
  adapters/  
src/  
  ... matching public modules ...  
gpu/  
  common/  
  cuda/  
    scans/  
    frontier/  
    distance/  
    drc/  
adapters/  
  kicad/  
tools/  
  apgar-cli/  
  replay-viewer/  
  benchmark/  
tests/  
  unit/  
  property/  
  differential/  
  integration/  
  corpus/  
benchmarks/  
  manifests/  
  synthetic/  
schemas/  
  board_ir/  
  replay/  
  telemetry/
```

Appendix B. Candidate Lifecycle State Machine

```
Generated  
-> normalized  
-> exact-validated  
  -> rejected (diagnostic retained)  
  -> accepted  
    -> stored
```

- > selected by zero or more worlds
- > legalization pending
 - > legal
 - > host validation
 - > committable
 - > host rejected
 - > legalization failed
- > pruned when unreferenced and dominated

Appendix C. Definition of Done for M1

- Board IR and schema documented and versioned.
- KiCad fixture adapter imports selected test boards.
- Exact geometry predicates have property and boundary tests.
- Sparse tiles and directional masks pass conservatism tests.
- CPU reference A* produces reproducible paths.
- At least one GPU generator matches CPU path cost on supported exact microcases.
- Candidate store records provenance, geometry, resources, metrics, and diversity.
- Exact candidate checker rejects every injected violation.
- Benchmark report includes time, VRAM, quality, and diversity.
- One replay bundle can reproduce a deliberately injected invariant failure.